Lucien Roberson
CS5600
HW6

Chapter 18:

Q1:
The output from the first command is massive. I know that as the address space increases with the page size held constant the number of pages will increase too.

```
vboxuser@Lucien:~/5600/hw6$ python3 paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
ARG seed 0
ARG address space size 1m
ARG phys mem size 512m
ARG page size 1k
ARG verbose True
ARG addresses -1


The format of the page table is simple:
The high-order (left-most) bit is the VALID bit.
  If the bit is 1, the rest of the entry is the PFN.
  If the bit is 0, the page is not valid.
Use verbose mode (-v) if you want to print the VPN # by
each entry of the page table.

Page Table (from entry 0 down to the max size)
  [       0]  0x8006104a
  [       1]  0x00000000
  [       2]  0x00000000
  [       3]  0x80033d4e
  [       4]  0x80026d2f
  [       5]  0x00000000
  [       6]  0x800743d0
  [       7]  0x80024134
```

Q2:
Page size increases linearly with a fixed page size so as we keep increasing page size the number of pages decreases.
Using large pages would increase internal fragmentation.
I haven't included photos because the output is so massive it wouldn't communicate the change in behavior.

Q3:
Running python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 0 -n 5 -c outputted

```
Page Table (from entry 0 down to the max size)
  [       0]  0x00000000
  [       1]  0x00000000
  [       2]  0x00000000
  [       3]  0x00000000
  [       4]  0x00000000
  [       5]  0x00000000
  [       6]  0x00000000
  [       7]  0x00000000
  [       8]  0x00000000
  [       9]  0x00000000
  [      10]  0x00000000
  [      11]  0x00000000
  [      12]  0x00000000
  [      13]  0x00000000
  [      14]  0x00000000
  [      15]  0x00000000

Virtual Address Trace
  VA 0x00003a39 (decimal:    14905) -->  Invalid (VPN 14 not valid)
  VA 0x00003ee5 (decimal:    16101) -->  Invalid (VPN 15 not valid)
  VA 0x000033da (decimal:    13274) -->  Invalid (VPN 12 not valid)
  VA 0x000039bd (decimal:    14781) -->  Invalid (VPN 14 not valid)
  VA 0x000013d9 (decimal:     5081) -->  Invalid (VPN 4 not valid)
```

While running python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 100 -n 5 -c outputted :

```
Page Table (from entry 0 down to the max size)
  [       0]  0x80000018
  [       1]  0x80000008
  [       2]  0x8000000c
  [       3]  0x80000009
  [       4]  0x80000012
  [       5]  0x80000010
  [       6]  0x8000001f
  [       7]  0x8000001c
  [       8]  0x80000017
  [       9]  0x80000015
  [      10]  0x80000003
  [      11]  0x80000013
  [      12]  0x8000001e
  [      13]  0x8000001b
  [      14]  0x80000019
  [      15]  0x80000000

Virtual Address Trace
  VA 0x00002e0f (decimal:    11791) --> 00004e0f (decimal    19983) [VPN 11]
  VA 0x00001986 (decimal:     6534) --> 00007d86 (decimal    32134) [VPN 6]
  VA 0x000034ca (decimal:    13514) --> 00006cca (decimal    27850) [VPN 13]
  VA 0x00002ac3 (decimal:    10947) --> 00000ec3 (decimal     3779) [VPN 10]
  VA 0x00000012 (decimal:       18) --> 00006012 (decimal    24594) [VPN 0]
```

Q4:

- -P 8 -a 32 -p 1024 -v -s 1 has way too small of pages
- -P 8k -a 32k -p 1m -v -s 2 is just right
- -P 1m -a 256m -p 512m -v -s 3 has way too large of pages

Q5:
The example to try out gives an expected output

```
vboxuser@Lucien:~/5600/hw6$ python3 paging-linear-translate.py -P 1k -a 16k -p 8k -v -n 5 -c
ARG seed 0
ARG address space size 16k
ARG phys mem size 8k
ARG page size 1k
ARG verbose True
ARG addresses -1

Error: physical memory size must be GREATER than address space size (for this simulation)
```

Ch 19:

Q1:
gettimeofday() resolution is in microseconds, so to get a relatively precise operation you should use something 100 times greater, like .1ms.
Q2:

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <sched.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "usage: tlb <numpages> <trials>\n");
        exit(1);
    }
    long numpages = atol(argv[1]);
    long trials   = atol(argv[2]);
    long pagesize = sysconf(_SC_PAGESIZE);
    long jump     = pagesize / sizeof(int);

    // pin to one CPU so migrations don't pollute the TLB measurement
    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(0, &set);
    sched_setaffinity(0, sizeof(set), &set);

    int *a = calloc(numpages, pagesize);   // calloc touches/zeros pages
    if (a == NULL) { perror("calloc"); exit(1); }

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (long t = 0; t < trials; t++)
        for (long i = 0; i < numpages * jump; i += jump)
            a[i] += 1;
    clock_gettime(CLOCK_MONOTONIC, &end);

    double ns = (end.tv_sec - start.tv_sec) * 1e9 + (end.tv_nsec - start.tv_nsec);
    printf("%ld pages: %.2f ns/access\n", numpages, ns / (trials * numpages));
```

```
vboxuser@Lucien:~/5600/hw6$ for n in 1 2 4 8 16 32 64 128 256 512 1024 2048 4096 8192; do ./tlb
 $n 1000000; done
1 pages: 2.17 ns/access
2 pages: 2.30 ns/access
4 pages: 2.97 ns/access
8 pages: 3.35 ns/access
16 pages: 6.01 ns/access
32 pages: 10.17 ns/access
64 pages: 11.55 ns/access
128 pages: 12.12 ns/access
256 pages: 11.62 ns/access
512 pages: 11.56 ns/access
1024 pages: 11.59 ns/access
2048 pages: 13.53 ns/access
4096 pages: 18.38 ns/access
8192 pages: 19.78 ns/access
```

Q3:

```python
import re; import sys; import matplotlib
matplotlib.use("Agg")  # no display needed; saves to PNG
import matplotlib.pyplot as plt

infile = sys.argv[1] if len(sys.argv) > 1 else "results.txt"
outfile = sys.argv[2] if len(sys.argv) > 2 else "tlb.png"

pages, times = [], []
with open(infile) as f:
    for line in f:
        nums = re.findall(r"\d+\.?\d*", line)
        if len(nums) >= 2:
            pages.append(int(nums[0]))
            times.append(float(nums[1]))

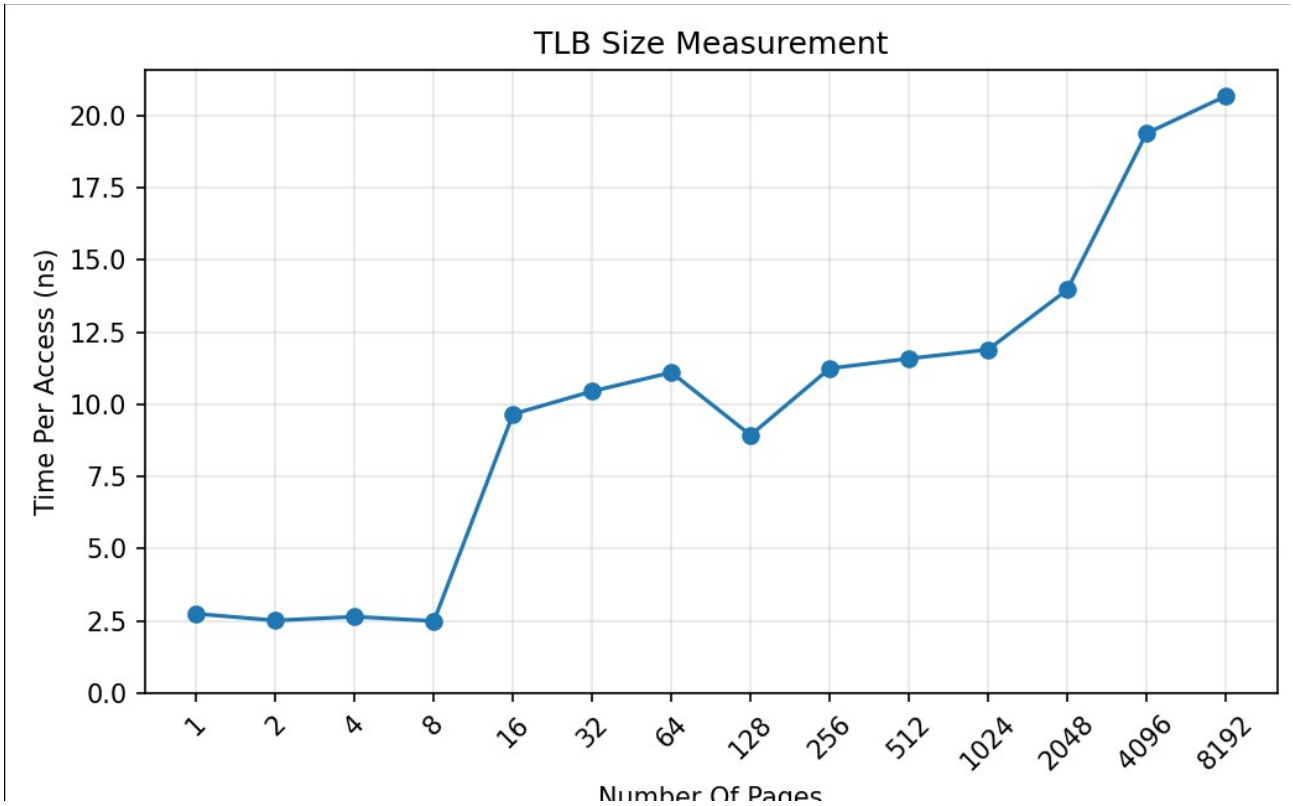
if not pages:
    sys.exit(f"no data parsed from {infile}")

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(pages, times, marker="o", color="tab:blue")

ax.set_xscale("log", base=2)
ax.set_xticks(pages)
ax.set_xticklabels([str(p) for p in pages], rotation=45)
ax.set_xlabel("Number Of Pages")
ax.set_ylabel("Time Per Access (ns)")
ax.set_title("TLB Size Measurement")
ax.set_ylim(bottom=0)
ax.grid(True, alpha=0.3)

fig.tight_layout()
fig.savefig(outfile, dpi=150)
print(f"saved {outfile} ({len(pages)} points)")
```

[Wrote 37 lines]



Q5:
You declare the array volatile so the compiler performs every read and write and can’t destroy the loop.

Q6:
You can pin the process with sched_setaffinity(). If you don’t the scheduler can migrate to another core that don’t have your caches or transitions.

Q7
The first touch of each page triggers a page fault which costs more than a TLB miss and would inflate the timings. Touch every page at once before starting the timer.